

Idea

Working concept: Beeline

One-liner

Beeline turns hours of course video into a prerequisite graph, then plays you the **shortest sequence of clips** that takes you from what you already know to the one concept you want to understand.

Problem

Educational video is linear, but knowledge is a graph.

A learner who wants to understand **one specific concept** buried in a lecture series has two bad options:

- Watch everything — hours of material for minutes of relevance.
- Skip ahead using search or chapter markers — and land mid-lecture in an explanation that silently assumes three things they haven't seen.

Nobody can easily answer:

- Where exactly is this concept *actually explained* (not just mentioned)?
- What does that explanation assume I already know?
- Which of those prerequisites can I safely skip?
- What order should I watch the remaining pieces in?
- Is anything my target depends on *never explained* in this corpus at all?

Search engines and chapter lists answer "where is the word said." None of them answer "what is the cheapest path to understanding."

Solution

Beeline ingests a video corpus and builds a **concept graph**: what each clip teaches, what it assumes, and which concepts require which.

The user types the concept they want. Beeline:

1. Resolves the query to a concept node in the graph.

2. Computes the full prerequisite closure of that concept.
3. Removes everything the user marks as already known — plus anything only needed for those.
4. Selects the **minimum-watch-time set of clips** that covers every remaining concept.
5. Orders the clips so prerequisites always play first.
6. Plays them as one continuous session, with the graph lighting up as you progress.

Each concept node is colored by state:

- **Blue:** on your path — a clip in your playlist covers it
- **Green:** already known — pruned, you'll never see it
- **Gray:** not needed for this target
- **Red:** a gap — required by your target but never explained anywhere in the corpus

Beeline does not decide whether you have *understood* anything. It guarantees **coverage and order**: every prerequisite of your target is explained by some clip in your path, prerequisites come first, and any gap the corpus can't cover is shown honestly instead of hidden.

The impossible question

Across 2 hours and 20 minutes of lectures, what is the minimum set of moments — in what order — that takes someone who already knows matrix multiplication to understanding attention, and which prerequisite does this corpus never actually teach?

No single tool can answer this.

Technology	What it proves
TwelveLabs	<i>Where</i> each concept is genuinely taught — across visuals, speech, audio, and on-screen math/slides/code — and where each video's natural chapter boundaries fall
OpenAI	<i>What</i> each segment teaches vs. silently assumes, which concept names are the same concept, and why the resulting path is ordered the way it is
Neo4j	<i>How</i> concepts depend on each other — the REQUIRES/EXPLAINS graph, the prerequisite closure, and the query-to-concept vector resolution
Strands Agents	<i>That the whole pipeline runs</i> — orchestrates ingestion (index → chapter → extract → load) and the query-time path agent as reliable tool-calling loops

Example corpus

Use **one coherent playlist**: the 3Blue1Brown *Neural Networks* series (7 videos, $\approx$ 2h20m total). Every video builds on the previous ones, so prerequisite edges are real, not imagined.

(If your hackathon requires openly licensed material, MIT OpenCourseWare lectures are the drop-in substitute — the plan doesn't change.)

#	Video (approx.)	Length
1	But what is a neural network?	~19 min
2	Gradient descent, how machines learn	~21 min
3	What is backpropagation really doing?	~14 min
4	Backpropagation calculus	~10 min
5	But what is a GPT?	~27 min
6	Attention in transformers	~26 min
7	How might LLMs store facts	~22 min

The extracted graph (excerpt) should look like:

attention REQUIRES embedding, softmax, dot product
embedding REQUIRES vectors, neural network
backpropagation REQUIRES gradient descent, neural network
gradient descent REQUIRES neural network, cost function
neural network REQUIRES matrix multiplication
softmax REQUIRES vectors
dot product REQUIRES vectors

With clips attached (example values):

V1 05:30–08:10 EXPLAINS neural network (0.90)
V2 03:15–06:40 EXPLAINS gradient descent (0.93), cost function (0.71)
V5 12:10–15:05 EXPLAINS embedding (0.91)
V5 18:20–20:00 EXPLAINS softmax (0.88)
V6 03:40–08:55 EXPLAINS attention (0.94), dot product (0.35)

Beeline result

Query: "attention" — nothing marked as known

7 clips · 24 minutes · ordered: matrix multiplication → neural network → dot product
→ softmax → embedding → attention Corpus: 2h 20m. You watch: **24 minutes**.

Same query — "I already know: linear algebra, neural networks" checked

4 clips · 12 minutes. Three subtrees pruned in front of the user's eyes; the counter drops live.

The honest discovery

positional encoding — Red. Attention's explanation references it, but no clip in this corpus ever teaches it. Beeline shows the gap instead of pretending the path is complete.

The live shrink is the wow moment. The red gap node is the credibility moment.

How the correlation works

The hard part is that the same concept has different names in different modalities:

Spoken: "backprop"
Slide title: "Backpropagation"
On-screen math: $\partial C / \partial w$ chain rule walkthrough
Chapter title: "The backward pass"

Beeline resolves them into one node:

Concept: backpropagation

- └─ heard in speech (TwelveLabs audio/speech)
- └─ read on slides (TwelveLabs on-screen text)
- └─ described in chapters (Pegasus segmentation)
- └─ merged by OpenAI (embedding similarity + alias table)

Canonicalization for the MVP: lowercase, embed every extracted concept name with OpenAI embeddings, merge pairs above ~0.85 cosine, and keep a small hand-written [aliases.json](#) as an override for anything the demo depends on. Do not build a sophisticated resolver.

Graph schema (Neo4j):

```
(:Video {id, title, youtube_url, duration})
(:Clip {id, start, end, duration, summary})-[:PART_OF]->(:Video)
(:Clip)-[:EXPLAINS {score}]->(:Concept {name, embedding, aliases})
(:Concept)-[:REQUIRES {confidence}]->(:Concept)
```

Prerequisite closure (query time):

```
MATCH (t:Concept {name: $target})-[:REQUIRES*0..5]->(p:Concept)
RETURN DISTINCT p
```

Clip selection: the closure minus known concepts is a weighted set-cover instance — NP-hard, so use the classic greedy: repeatedly pick the clip with the best (needed concepts covered ÷ duration) ratio. Greedy carries a proven $\sim \ln(n)$ guarantee, and at this scale you can verify it against the exact ILP optimum in milliseconds. *"Our clip selection is NP-hard; we run the greedy approximation and verified it lands within a few percent of the ILP optimum on our corpus"* is one sentence of real rigor for judges.

Ordering: topological sort of the pruned concept DAG; each clip plays at the earliest position of the concepts it covers; ties break by (video, timestamp).

JSON contract

Graph mode — what the frontend consumes:

```
{
  "query": "attention",
  "mode": "graph",
  "target_concepts": ["attention"],
  "known": ["linear algebra", "neural network"],
  "needed_concepts": ["dot product", "softmax", "embedding", "attention"],
  "playlist": [
    {
      "clip_id": "v6_c2",
      "video_id": "V6",
      "video_title": "Attention in transformers",
      "youtube_url": "https://www.youtube.com/watch?v=...",
      "start_seconds": 220,
      "end_seconds": 535,
      "covers": ["attention"],
      "why": "The core explanation of attention. Plays last because it assumes softmax and embeddings, which your path covers first."
    }
  ],
  "gaps": ["positional encoding"],
  "total_corpus_seconds": 8400,
  "watch_seconds": 720,
```

```
"narration": "You'll start with dot products (2 min) because attention scores are dot products..."
}
```

The player interaction

The user sees one page: query box, the graph, the player.

Typing **"attention"** and pressing **Build my path**:

1. The graph renders — target pulses, path nodes turn blue, pruned nodes fade green, gaps glow red.
2. The playlist strip appears: 4 clip cards with durations, and the counter — **"2h 20m → 12m."**
3. The player auto-plays clip 1. A banner explains: *"Now: softmax — because attention weights are a softmax."*
4. Each clip stops at its end boundary and auto-advances. No seeking, no hunting.
5. The user ticks **"I already know: embeddings"** — the path recomputes, a subtree fades, the counter drops again.
6. Clicking any node jumps the player to the clip that covers it and shows its evidence: which video, which chapter, EXPLAINS score, what it assumes.

Kill shot

Add one toggle: **Graph path** vs. **Search only**.

Graph path (TwelveLabs + OpenAI + Neo4j + Strands):

Ordered playlist, 4 clips, 12 minutes. Coverage: **4/4** needed concepts, prerequisites first, 1 gap disclosed.

Search only (TwelveLabs search, scoped to the raw query):

1 moment found where "attention" is discussed — minute 4 of video 6.
Prerequisites required: **unknown**. Skippable material: **unknown**. Watch order: **unknown**. Coverage: **1/4**. You land mid-lecture in an explanation that assumes three concepts you haven't seen.

Your closing line:

Search shows where a concept is said. Only the graph shows what it takes to understand it — and what you're allowed to skip.

How each technology is load-bearing

Technology

Essential responsibility

Strands Agents	Orchestrates the video-to-graph pipeline: an Ingestion Agent drives index → chapter → extract → canonicalize → load per video with retries and caching; a Path Agent turns a user query into resolve → closure → cover → order → narration through tool calls
OpenAI	Reasons over extracted context: classifies what each chapter teaches vs. assumes, proposes REQUIRES edges, merges duplicate concept names via embeddings, and narrates why the path is ordered the way it is
TwelveLabs	Understands vision, audio, speech, and on-screen content: Marengo powers moment search across all modalities (including the search-only kill shot); Pegasus produces chapter segmentation and per-chapter descriptions that feed extraction
Neo4j	Models entities, scenes, tags, and their connections: Concept/Clip/Video nodes, REQUIRES/EXPLAINS/PART_OF edges, the vector index that resolves free-text queries to concepts, and the closure traversal that defines the path

Remove any one of the four and a demo step visibly dies: no Strands → the pipeline is a pile of scripts with no reliable loop; no OpenAI → chapters but no edges; no TwelveLabs → no clips and no kill shot; no Neo4j → no closure, no pruning, no path.

What Beeline does not claim

- It does not certify that the user has understood or mastered anything.
- It does not grade, test, or estimate ability.
- It claims exactly one thing: *every prerequisite of your target is covered by a clip in your path, in a valid order — and where the corpus can't cover one, you are told.*
- Gaps are displayed, never silently dropped.

Five-hour MVP

Do **not** ingest arbitrary URLs live on stage or build accounts, quizzes, or mastery tracking.

Build only:

1. One pre-indexed playlist (5–7 videos, indexed before the clock starts if rules allow)
2. One concept graph (~50–80 concepts) in Neo4j Aura Free
3. One Strands ingestion agent that produced that graph end-to-end
4. One query box → one ordered playlist
5. One seamless player with auto-advance and per-clip "why" banners

6. One "I already know X" checklist + live minutes counter
7. One graph visualization with blue/green/gray/red states
8. One Graph-path vs. Search-only kill-shot toggle
9. One red gap node in the demo path

Avoid:

- multiple corpora or corpus switching
- user accounts, history, spaced repetition
- quiz or flashcard generation
- mastery/understanding estimation
- live ingestion of user-pasted URLs
- transcript editors, note-taking, mobile layouts
- fine-tuning anything

Five-hour clock

Hour	P1 Frontend	P2 Ingestion	P3 Graph	P4 Integration
0–1	Layout + player vs. mock JSON	Index created, uploads running, chapters flowing	Aura up, schema + loader vs. hand-written mini-graph	Repo, contract, FastAPI skeleton
1–3	Graph viz + checklist + counter	Extraction + canonicalize → <code>graph_payload.json</code>	Cycle-break + closure + set cover + topo, CLI works	<code>/api/path</code> wraps P3, Strands Path Agent + narration
3–4	Kill-shot toggle, polish	Backfill remaining videos, cache complete	Canned queries verified, gap detection	Wire frontend to live API, search-only mode
4–5	—	—	—	Everyone: rehearse, cache demo responses, fix the one thing that breaks

Best product name options

- **Beeline**
- PathCut
- ClipRoute
- MinuteMap

- WatchLess
- PrereqGraph

Beeline remains the clearest name because it communicates the promise in one word: the shortest possible line from where you are to the thing you want to understand.

Final pitch line

Nobody needs another two-hour lecture. They need the twelve minutes of it that unlocks the thing they're stuck on — and a straight line to it.

Shred Repo

Set up the shared repository structure for the Beeline hackathon MVP.

First inspect the existing repository. Create only missing folders and files. Do not overwrite, redesign, or delete anything that already exists.

Create this structure:

```
beeline/
├── frontend/
│   └── README.md
├── ingestion/
│   └── README.md
├── graph/
│   └── README.md
├── integration/
│   └── README.md
├── shared/
│   ├── types.ts
│   └── sample_path.json
├── data/
│   └── cache/
│       └── .gitkeep
├── README.md
└── .env.example
```

Folder ownership — document these responsibilities in the root README.md:

- **frontend/** — Person 1: player, graph visualization, known-concepts checklist, counters, kill-shot toggle
- **ingestion/** — Person 2: TwelveLabs indexing + Strands ingestion agent → **graph_payload.json**
- **graph/** — Person 3: Neo4j schema, loader, path engine (closure, set cover, ordering)
- **integration/** — Person 4: FastAPI **/api/path**, Strands Path Agent, demo
- **shared/** — the response contract everyone codes against
- **data/cache/** — every external API response is cached here by request hash; committed cache = offline demo

Shared type — add this to **shared/types.ts**:

```
export type NodeState = "on_path" | "known" | "not_needed" | "gap";
```

```
export interface ClipSegment {
  clip_id: string;
  video_id: string;
  video_title: string;
  youtube_url: string;
  start_seconds: number;
  end_seconds: number;
  covers: string[];
  why: string;
}
```

```
export interface PathResult {
  query: string;
  mode: "graph" | "search_only";
  target_concepts: string[];
  known: string[];
  needed_concepts: string[];
  playlist: ClipSegment[];
  gaps: string[];
  total_corpus_seconds: number;
  watch_seconds: number;
  narration: string;
}
```

Sample response — add one valid example to **shared/sample_path.json** using the same structure (use the "attention" example from the plan).

Folder README files — each contains only: folder owner, responsibility, expected input, expected output, how to run it once implemented.

Environment file — add empty placeholders only to `.env.example`. Never commit a real key, anywhere, ever:

```
TWELVELABS_API_KEY=  
TWELVELABS_INDEX_ID=  
OPENAI_API_KEY=  
NEO4J_URI=  
NEO4J_USERNAME=  
NEO4J_PASSWORD=
```

Do not create application logic, install packages, add authentication, create APIs, or build any product features.

When finished, print the repository tree and list which files were created. Then stop.

Process

Step	What happens
1	One person creates the GitHub repo.
2	That person runs the Shred Repo prompt once and pushes the structure to main.
3	Everyone clones and pulls latest main.
4	Each person creates their own branch.
5	Give each person the shared repo rules, then their individual prompt below.
6	Each person works only inside their assigned folder and pushes their branch.
7	Person 4 merges and connects everything.

Prompt distribution

Person	Prompts to give
Person 1	Repo rules + Frontend prompt

Person 2 Repo rules + TwelveLabs/Strands ingestion prompt

Person 3 Repo rules + Neo4j graph + path engine prompt

Person 4 Repo rules + Integration prompt

Branches

person-1-frontend
person-2-ingestion
person-3-graph
person-4-integration

Important: create the structure once, push to main, everyone pulls. Do not have four people scaffold independently.

P1 — Frontend (UI/UX)

Build the frontend UI for Beeline, a hackathon MVP that plays the shortest sequence of video clips needed to understand one concept.

Keep the implementation simple. Do not add a backend, authentication, settings, notifications, analytics, or extra screens.

Product

Beeline shows a concept graph extracted from a lecture playlist and plays an ordered, minimal playlist of clips for the concept the user asks about. The UI should feel like a modern learning/engineering tool: player as the hero, graph as the proof, numbers as the punchline.

Page layout — one desktop-first page.

Left sidebar:

- Beeline logo/name
- Corpus name: "Neural Networks — 7 lectures · 2h 20m"
- Query input + button: "**Build my path**"
- Mode toggle: "**Graph path**" / "**Search only**"

- **"I already know..."** checklist (concept names with checkboxes); changing any checkbox recomputes the path
- Playlist list: each clip with title, duration, covered concepts, and a status dot (upcoming / playing / done)

Main area:

- YouTube IFrame Player. Requirements: `seekTo(start_seconds)`, poll current time, hard-stop at `end_seconds`, auto-advance to the next clip
- Banner above the player: *"Now: softmax — because attention weights are a softmax"* (use each clip's `why`)
- The counter, large: **"2h 20m → 12m"** (`total_corpus_seconds` vs. `watch_seconds`), animating when the path changes

Right panel — the graph:

- Force-directed concept graph (react-force-graph)
- Node states: blue `on_path`, green faded `known`, gray `not_needed`, red `gap`
- Currently-playing concept pulses; completed path nodes get a check
- Clicking a node seeks the player to the clip covering it and shows a small evidence card: video, timestamps, EXPLAINS score, what it assumes
- Red gap nodes show: *"Required, but never explained in this corpus"*

Demo data — keep it all in a local JSON file matching `shared/types.ts`:

- Query "attention", known = [] → 7 clips, 24 min
- Query "attention", known = ["linear algebra", "neural network"] → 4 clips, 12 min
- One gap: "positional encoding" (red)
- Search-only variant: 1 clip, `needed_concepts` empty, narration explaining that prerequisites are unknown

Kill-shot interaction

When **"Search only"** is selected:

- Graph loses all path coloring except one highlighted node
- Playlist collapses to one clip
- Counter is replaced with: *"Coverage: 1/4 — prerequisites unknown"*
- Narration: *"The current moment is visible, but what it assumes, what you can skip, and what order to watch cannot be determined."*

This toggle must visibly demonstrate why the graph matters.

Visual direction

- Clean learning-tool aesthetic: off-white background, dark text, one accent color
- Blue/green/gray/red reserved exclusively for node states
- Dense enough to feel real, readable in a 90-second demo
- Avoid gradients, glassmorphism, oversized headings, generic AI sparkle

Technical requirements

- React, react-force-graph, YouTube IFrame Player API
- All demo data local; the app must fully work with no backend
- One page only; laptop-demo responsive
- Code against `shared/types.ts` exactly

The final result should make this flow obvious: *type a concept* → *watch the graph light up* → *press play* → *tick "I already know"* → *watch the path shrink* → *flip to Search only* → *watch the answer degrade*.

Done when the full flow runs on local JSON alone.

P2 — Ingestion (TwelveLabs + Strands Agents)

You own TwelveLabs + the Strands ingestion agent for Beeline.

Goal

Produce one file, `graph_payload.json`, containing concepts, REQUIRES edges, clips, and EXPLAINS edges for at least 5 of the 7 corpus videos — generated by a Strands agent run, with every external call cached.

TwelveLabs

- Create one index with Marengo (search) and Pegasus (generation) enabled
- Split any video over ~20 minutes into ~10-minute chunks with `ffmpeg -c copy` (seconds, splits at keyframes) and upload all chunks concurrently — per-chunk indexing collapses the wait
- Record each chunk's start offset; every timestamp you emit must be corrected back to the original video's timeline
- Poll tasks until ready; process videos in demo order (video 6 and 5 first — the attention path)
- Per chunk, request Pegasus **chapters** → `{start, end, title, summary}`. These become Clip records

- Fallback if chapters disappoint: fixed ~90s windows aligned to caption sentence boundaries (yt-dlp captions), so you are never blocked on indexing

OpenAI

- Per chapter, one structured-output call returning exactly:

```
{
  "teaches": [{"concept": "softmax", "score": 0.88}],
  "assumes": ["vectors", "exponential function"]
}
```

- Concepts: lowercase noun phrases, ≤ 4 words
- Canonicalize: embed every concept name, merge pairs above ~0.85 cosine, then apply `aliases.json` overrides (hand-written, demo-critical merges only)
- Emit edges: chapter teaches C and assumes P $\rightarrow$ C **REQUIRES** P with confidence = number of chapters asserting it

Strands Agents

- Wrap each step as a `@tool` function: `upload_video`, `await_ready`, `get_chapters`, `extract_concepts`, `canonicalize`, `append_payload`
- One **Agent** (OpenAI model provider) with those tools and a system prompt that drives: *for each video $\rightarrow$ wait $\rightarrow$ chapters $\rightarrow$ extract $\rightarrow$ append*
- Every tool caches its API response to `data/cache/` keyed by request hash; a re-run must be a no-op that rebuilds `graph_payload.json` from cache
- Keep it to one agent with tools. Do not build swarms, A2A, sessions, or deployment infrastructure

Return this exact payload shape

```
{
  "videos": [{"id": "V6", "title": "Attention in transformers", "youtube_url": "...", "duration": 1560}],
  "clips": [{"id": "v6_c2", "video_id": "V6", "start": 220, "end": 535, "summary": "..."}],
  "concepts": [{"name": "attention", "aliases": ["self-attention"]}],
  "explains": [{"clip_id": "v6_c2", "concept": "attention", "score": 0.94}],
  "requires": [{"from": "attention", "to": "softmax", "confidence": 3}]
}
```

Done when

- `graph_payload.json` covers ≥ 5 videos, ≥ 50 concepts, both edge types

- "attention", "softmax", "embedding", "backpropagation" exist as single canonical nodes each
- Re-running the agent costs zero API calls
- P3 can load the file without asking you anything

Do not build a UI, a vector database, arbitrary-URL ingestion, or fine-tuned models.

P3 — Graph + path engine (Neo4j)

You own Neo4j and the path algorithm for Beeline.

Goal

Given `graph_payload.json`, load the graph and expose one function that turns `(query, known[])` into a valid `PathResult`.

Neo4j

- Aura Free instance; uniqueness constraints on `Concept.name`, `Clip.id`, `Video.id`
- Loader: idempotent `MERGE`s from the payload
- Vector index on `Concept.embedding` (embed names with OpenAI at load time)
- Cycle-break at load: pull `REQUIRES` edges into `networkx`, find cycles, delete the lowest-confidence edge in each, write deletions back. The graph must be a DAG before any query runs

Path engine (Python, importable + CLI):

1. **Resolve**: embed the query → vector index → top concept (top-2 if scores are close; return both as `target_concepts`)
2. **Closure**: `MATCH (t:Concept {name:$t})-[:REQUIRES*0..5]->(p) RETURN DISTINCT p` — cap depth at 5
3. **Prune known**: BFS from the target that stops at known nodes; drop known nodes and anything reachable only through them
4. **Select clips**: greedy weighted set cover — repeatedly take the clip maximizing (sum of `EXPLAINS` scores over still-needed concepts ÷ clip duration); any concept no clip covers goes to `gaps`
5. **Order**: topological sort of needed concepts (Kahn); each clip sorts by the earliest topo position among concepts it covers; ties by (video, start)
6. **Emit** a `PathResult` matching `shared/types.ts`, including `watch_seconds` and `total_corpus_seconds`

CLI

```
python path.py "attention" --known "neural network,linear algebra"
```

prints the JSON.

Done when

- The three canned queries ("attention", "backpropagation", "how does a network learn") return valid, deterministic playlists
- `watch_seconds` < 25% of corpus on each
- Adding a known concept never increases watch time
- The positional-encoding gap appears in `gaps`, not in `playlist`
- Optional flex: verify greedy against the exact ILP optimum (PuLP) on the canned queries and record the gap %

Do not use GDS plugins, write to the graph at query time, add rerankers, or build an API — Person 4 wraps you.

P4 — Integration + demo

You own full-system integration and the 90-second demo for Beeline.

Goal

Connect P1's frontend, P2's graph payload, and P3's engine into one live flow, wrapped in a Strands Path Agent.

API

Expose one endpoint:

```
POST /api/path { "query": str, "known": [str], "mode": "graph" | "search_only" }
```

It must:

1. If `mode = "graph"`: run a Strands `Agent` whose tools are P3's functions (`resolve_concept`, `prereq_closure`, `select_clips`) plus a narration step — one OpenAI call that writes the `narration` and per-clip `why` strings from the computed path (never invents clips or reorders them)

2. If `mode = "search_only"`: call TwelveLabs search with the raw query, return the top moment as a one-clip `PathResult` with empty `needed_concepts`, the degraded narration, and no gaps analysis
3. Return JSON matching `shared/types.ts` exactly
4. Cache every response; a `--canned` flag serves the three rehearsed queries from disk with zero live calls

Your tasks

1. Wire "Build my path" to `/api/path`
2. Map returned concepts/clips onto P1's graph states and playlist
3. Make the known-concepts checklist re-query and animate the counter
4. Make the mode toggle work end-to-end
5. Test the full flow on venue-grade wifi (i.e., on your phone's hotspot)
6. Script and rehearse the 90-second demo

The 90-second demo

1. "This playlist is 2 hours 20 minutes. I want to understand one thing: attention." → type it, build
2. Graph lights up; counter reads **24 minutes**
3. "But I already know linear algebra and basic neural nets." → two checkboxes → **12 minutes**, subtrees fade
4. Press play; let one clip auto-advance across a boundary — prove seamlessness
5. Point at the red node: "And Beeline is honest — this corpus never teaches positional encoding."
6. Flip to **Search only**: "Here's what search alone gives you." → coverage 1/4, everything unknown
7. Closing line: *"Search shows where a concept is said. Only the graph shows what it takes to understand it."*

Done when

- One button runs the live investigation end-to-end
- Checkbox → visible shrink works without reload
- Search-only visibly degrades the answer
- The demo runs start-to-finish from cache with the network cable pulled

Do not redesign the UI, add features, or change the shared JSON contract.

Seeding the corpus

For a 5-hour MVP, do not build general ingestion. Index one playlist that tells one coherent story, and pre-compute everything the demo touches.

Corpus

- 3Blue1Brown *Neural Networks* (7 videos, $\approx$ 2h20m). Coherent prerequisite structure, famous, visually legible on a projector
- Openly licensed substitute if required: one MIT OCW lecture series

Before the clock starts (if rules allow — email the organizers; most restrict code, not data prep)

- TwelveLabs index created, all videos chunked, uploaded, and indexed
- yt-dlp captions downloaded as the fallback text source
- `data/cache/` populated so hour 0 starts warm
- If TwelveLabs is a sponsor booth: ask them to bump concurrent-task limits — concurrency is the real cap on chunked upload speed

What the extractor should produce (per chapter):

```
{ "clip": "v5_c4", "teaches": [{"concept": "embedding", "score": 0.91}], "assumes": ["vectors", "neural network"] }
```

Alias mapping — hard-code the demo-critical merges in `aliases.json`:

```
{
  "backpropagation": ["backprop", "the backward pass", "chain rule walkthrough"],
  "attention": ["self-attention", "attention head", "attention mechanism"],
  "embedding": ["word embedding", "embedding vector", "token embedding"],
  "neural network": ["network", "multilayer perceptron", "mlp"]
}
```

Canned queries (rehearse exactly these three):

1. "attention" — the hero path + the shrink + the gap
2. "backpropagation" — a second, different-shaped path (proves it's not hard-coded)
3. "how does a network learn" — fuzzy phrasing that vector-resolves to *gradient descent* (proves query resolution is real)

Minimal artifact checklist

- 1 TwelveLabs index · 7 videos (~16 chunks)
- ~50 Pegasus chapters → ~50 Clip nodes
- 50–80 canonical concepts · 100–150 REQUIRES edges after confidence pruning

- 1 `aliases.json` · 1 `graph_payload.json`
- 3 canned queries cached end-to-end
- 1 red gap node that appears on the hero query

With roughly that much seeded, the demo convincingly shows multimodal understanding, LLM reasoning, agent orchestration, and a graph that earns its place — without spending the five hours on data plumbing.